

The Comprehensive Guide to Controllers

هذا الدليل مرجع عام لفهم وبناء أي Controller ضمن مشاريع ASP.NET Core Web API. الأمثلة مبنية على الـ Controllers الموجودة في مشروعك، لكن الشرح يركّز أولاً على المفهوم العام ثم يربطه بأمثلة حقيقية.

1. دور ووظيفة الـ Controller

الـ Controller هو نقطة الاستقبال المنظمة للطلبات القادمة من التطبيق الأمامي أو أدوات الاختبار. وظيفته ليست تنفيذ كل العمل بنفسه، بل تنظيم مسار الطلب: يحدد الـ Action المناسبة، يقرأ البيانات، يستدعي الجهة المسؤولة عن التنفيذ، ثم يعيد النتيجة بصيغة JSON مع حالة HTTP مناسبة.

```
Client → HTTP Request → Controller Action → DbContext or Service → Database →  
Controller → JSON Response
```

مثلاً، MenuItemController مسؤول عن التعامل مع عناصر الطعام: القراءة والإضافة والتعديل والحذف. أما OrderHeaderController فمسؤول عن الطلبات الرئيسية.

لماذا لا نضع منطق أعمال معقداً داخله؟

من الأفضل أن يبقى الـ Controller قصيراً وواضحاً. منطق مثل التحقق من صحة المعرف، استدعاء قاعدة البيانات، أو إعادة 404 Not Found مناسب داخله. أما حساب الخصومات والضرائب، فحص المخزون، الدفع، الإشعارات، وقواعد تغيير حالة الطلب، فيصبح أفضل داخل خدمة مثل OrderService عندما يكبر المشروع.

```
Receive request → validate data → call service → return response
```

2. الـ Imports أو Using Statements

الـ using يسمح للملف باستخدام أنواع وكلاسات موجودة في أجزاء أخرى من المشروع أو في المكتبات.

Microsoft.AspNetCore.Mvc

```
using Microsoft.AspNetCore.Mvc;
```

يوفر الأساسيات الخاصة بالـ Web API مثل `ControllerBase` و `Controller` و `[ApiController]` و `[Route]` و `[HttpGet]` والاستجابات مثل `Ok()` و `BadRequest()` و `NotFound()`.

MangoFusion_API.Data

```
using MangoFusion_API.Data;
```

يحتاجه أي `Controller` يتعامل مع `ApplicationDbContext`. يظهر ذلك في `MenuItemController` و `OrderHeaderController` و `OrderDetailsController`.

MangoFusion_API.Models

```
using MangoFusion_API.Models;
```

يوفر نماذج البيانات الرئيسية مثل `MenuItem` و `OrderHeader` و `OrderDetail` و `ApplicationUser` و `ApiResponse`.

MangoFusion_API.Models.Dto

```
using MangoFusion_API.Models.Dto;
```

يوفر كائنات `DTO` المخصصة لشكل البيانات المتبادلة مع العميل، مثل `LoginRequestDTO` و `RegisterRequestDTO` و `MenuItemCreatedDTO` و `OrderHeaderCreatedDTO`. استخدام `DTO` يحدد بدقة ما يسمح للعميل بإرساله.

Microsoft.EntityFrameworkCore

```
using Microsoft.EntityFrameworkCore;
```

تحتاجه عند استعمال مزايا Entity Framework Core مثل Include() و ThenInclude() و FirstOrDefaultAsync() و SaveChangesAsync(). في OrderHeaderController يستخدم لجلب الطلب مع تفاصيله وعناصر القائمة المرتبطة بها.

Authorization و Identity

```
using Microsoft.AspNetCore.Identity;  
using Microsoft.AspNetCore.Authorization;
```

يستخدم AuthController خدمات UserManager<ApplicationUser> و RoleManager<IdentityRole> لإدارة المستخدمين والأدوار. ويستخدم AuthTestController السمة [Authorize] لحماية المسارات.

MangoFusion_API.Utility

```
using MangoFusion_API.Utility;
```

يمنح الوصول إلى القيم المشتركة داخل SD، مثل أسماء الأدوار وحالات الطلب.

3. مفاهيم OOP المستخدمة

Inheritance الوراثة

ال Controller يرث من كلاس جاهز يوفر قدرات ال Web API. الصيغة الأنسب عادة لمشروع يعيد بيانات فقط هي:

```
public class ProductsController : ControllerBase
```

الكلاس ControllerBase يوفر أدوات مثل Ok() و BadRequest() و NotFound(). في مشروعك الحالي، ال Controllers ترث من Controller:

```
public class MenuItemController : Controller
```

الكلاس Controller يرث من ControllerBase ويضيف دعماً للصفحات. هذا يعمل بصورة صحيحة، لكن ControllerBase أوضح دلاليّاً إذا كان المشروع API فقط.

مثال من مشروعك:

```
[ApiController]
[Route("api/[controller]")]
public class OrderHeaderController : Controller
```

حقن الاعتماديات Dependency Injection

بدلاً من أن ينشئ الـ Controller اتصال قاعدة البيانات أو الخدمات بنفسه، يستقبلها جاهزة في الـ Constructor.

```
public OrderHeaderController(ApplicationDbContext db)
{
    _db = db;
}
```

هذا مثال حقيقي من OrderHeaderController. الإطار ينشئ ApplicationDbContext وفق الإعدادات ثم يمرره للـ Controller.

ومثال أوسع من AuthController:

```
public AuthController(
    UserManager<ApplicationUser> userManager,
    RoleManager<IdentityRole> roleManager,
    IConfiguration configuration)
```

هنا يطلب الـ Controller الخدمات المناسبة لوظيفته، بدلاً من التعامل مع المستخدمين وكلمات المرور وجدول الهوية يدوياً.

4. متى ننشئ Controller جديداً؟

أنشئ Controller جديداً عندما يظهر مورد جديد أو مسؤولية مستقلة في النظام.

- MenuItemController: عناصر القائمة.
- OrderHeaderController: الطلبات.
- OrderDetailsController: تفاصيل الطلب.
- AuthController: التسجيل وتسجيل الدخول.

إذا أردت جلب عنصر واحد حسب المعرّف، أضف Action إلى MenuItemController لأنه يتعامل مع المورد نفسه. أما إن أضفت الدفع أو الكوبونات، فمن المنطقي إنشاء PaymentController أو CouponController.

Same resource and responsibility? Yes → add an Action to the existing Controller No
→ create a new Controller

5. المتغيرات الأساسية داخل الـ Controller

ApplicationDbContext

```
private readonly ApplicationDbContext _db;
```

يوفر الوصول إلى البيانات عبر خصائص مثل _db.MenuItems و_db.OrderHeaders و_db.OrderDetails.

الخدمات Services

قد يحتاج الـ Controller خدمات متخصصة. في مشروعك، يستخدم AuthController كلاً من UserManager وRoleManager. كما يحتاج MenuItemController إلى IWebHostEnvironment للتعامل مع مسار حفظ الصور.

ApiResponse

يستخدم مشروعك ApiResponse لتوحيد شكل الاستجابة: النتيجة، حالة النجاح، ورسائل الخطأ.

لماذا private readonly؟

private يعني أن المتغير لا يستخدم خارج الـ Controller. وreadonly يعني أن قيمته تعين مرة واحدة داخل الـ Constructor ولا تتغير بعد ذلك. هذا يجعل الاعتمادات ثابتة وواضحة.

6. خطوات بناء أي Action

1. تحديد المورد والعملية

اسأل: ما البيانات التي أتعامل معها؟ وماذا أريد أن أفعل بها؟ إذا كانت المهمة عرض عناصر الطعام، فالمورد هو MenuItem والمكان المناسب هو MenuItemController.

2. اختيار HTTP Method

العملية	النوع	مثال من المشروع
قراءة بيانات	GET	جلب عناصر القائمة أو الطلبات
إنشاء مورد	POST	إنشاء طلب في OrderHeaderController
تعديل مورد	PUT	تعديل تقييم في OrderDetailsController
حذف مورد	DELETE	حذف عنصر في MenuItemController

3. تصميم Route

يجب أن يعبر الرابط عن المورد بوضوح:

```
GET    /api/MenuItem
GET    /api/MenuItem/5
POST   /api/OrderHeader
PUT    /api/OrderDetails/10
```

المعرّف يوضع عادة في الرابط، أما بيانات الإنشاء أو التعديل فتأتي في Request Body، أو في Form عند رفع ملف.

4. تحديد شكل البيانات الداخلة

استخدم DTO عندما تريد تحديد الحقول التي يسمح للعميل بإرسالها، مثل LoginRequestDTO و OrderHeaderCreateDTO و MenuItemCreateDTO.

5. التحقق من البيانات والصلاحيات

قبل العملية، تحقق من صحة البيانات، وجود المورد، وصلاحيّة المستخدم. إذا لم يوجد الطلب أو العنصر، أعد 404 Not Found بدلاً من نجاح فارغ.

6. تنفيذ العملية

استخدم ApplicationDbContext لبيانات التطبيق، و UserManager للمستخدمين وكلمات المرور، و RoleManager للأدوار، و Services عندما يكبر منطق الأعمال.

7. اختيار Return Type ونتيجة HTTP

غالباً ستعيد IActionResult أو <ApiResponse> ActionResult. النوع الأول يناسب الحالات التي يمكن أن تعيد فيها Ok() أو NotFound() أو BadRequest(). النوع الثاني ينسجم مع نمط ApiResponse في مشروعك.

الاستجابة	الحالة
200 OK	قراءة أو تعديل ناجح
201 Created	إنشاء مورد جديد
400 Bad Request	بيانات غير صالحة
401 Unauthorized	مستخدم غير مسجل أو توكن غير صالح
403 Forbidden	مستخدم بلا صلاحية
404 Not Found	مورد غير موجود

في MenuItemController، استخدم CreatedAtRoute(...) بعد إنشاء عنصر مثال جيد؛ فهو يعيد 201 Created ويشير إلى رابط المورد الجديد.

قائمة فحص سريعة

1. What is the resource? 2. What operation is needed? 3. Which HTTP method fits? 4. What route is clear? 5. Is a DTO needed? 6. Are authorization checks needed? 7. Which DbContext or Service performs the work? 8. Which HTTP status fits each outcome?

7. أخطاء شائعة وملاحظات من المشروع الحالي

منطق أعمال كبير داخل ال Controller

في OrderHeaderController توجد عملية إنشاء الطلب ثم إنشاء تفاصيله وقواعد لتغيير الحالة. هذا مفهوم في حجم المشروع الحالي، لكن عند زيادة القواعد يفضل نقلها إلى OrderService.

استخدام نموذج قاعدة البيانات مباشرة

استخدام DTO في مشروعك لعمليات الإنشاء والتعديل خطوة جيدة. لا تسمح للعميل بإرسال كائن كامل مثل ApplicationUser أو OrderHeader إذا كان يحتاج عدداً محدوداً من الخصائص فقط.

حماية البيانات المرتبطة بالمستخدم

يعرض AuthTestController مثالاً على [Authorize] وتقييد الدور. عند بناء عمليات لطلبات المستخدم، لا تعتمد على userId الذي يرسله العميل وحده؛ تحقق من هوية المستخدم الموجودة داخل JWT حتى لا يطلب مستخدم بيانات مستخدم آخر.

ترتيب Middleware للمصادقة

في ملف Program.cs، الترتيب المعتاد هو:

```
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

المصادقة تحدد هوية المستخدم أولاً، ثم التفويض يقرر هل يملك صلاحية الوصول.

المزج بين async والانتظار المتزامن

في AuthController يوجد استخدام مثل Result. أو GetAwaiter().GetResult() داخل عمليات غير متزامنة. عند كتابة عمليات جديدة، فضل await مع عمليات قاعدة البيانات وخدمات الهوية.

التعامل مع الملفات المرفوعة

يتعامل MenuItemController مع صور العناصر. عند بناء عملية رفع ملفات، تحقق من النوع والحجم، ولا تثق باسم الملف القادم من العميل، واستخدم اسماً آمناً وفريداً عند الحفظ.

الخلاصة

ال Controller الجيد لا يقاس بعدد الأسطر، بل بوضوح مسؤوليته:

```
Understand the HTTP request → validate data and authorization → call the appropriate
DbContext or Service → return a clear HTTP response
```

عند بناء أي Action جديدة، ابدأ بالمورد والمسؤولية، ثم حدد نوع الطلب والرابط وال DTO والصلاحيات،
وبعدها نفّذ العملية وأعد الاستجابة المناسبة.